Indian Institute of Technology Guwahati

CVE-Guided Typestate Analysis for Recurring Vulnerability Detection in OpenSSL

Detailed Methodology, Implementation, Evidence, and Learning Report

Updated through 16 October 2026

Project team	[Rutvij Wagh , Udayraj Choudhary]
Member A / Roll No.	[230101113]
Member B / Roll No.	[230101107]
Project root (WSL)	`/mnt/d/CVE-Typestate-Project/`
Report purpose	Project understanding and Progress Meeting 1 preparation

How to Use This Report

This document has two purposes. First, it explains the work completed so far in an orderly, technical format suitable for a progress meeting. Second, it is a learning guide for understanding why the project was selected, what each vulnerability teaches us, how the checker works, what the tests demonstrate, and what still needs to be done.

Read Chapters 1–4 first if you need to explain the research idea and project selection. Chapters 5–9 describe the selected CVEs, source setup, rules, and test results. Chapters 10–13 cover the newer benchmark, recurrence investigation, limitations, and remaining work. The appendices collect commands, file inventories, definitions, and references.

What can be said confidently now? Three CVE-specific rules have been implemented in an initial Python checker. The combined source-pattern regression script distinguished all three vulnerable/fixed source pairs. A separate 16-case hand-written snippet benchmark also passed, but its 100% metrics apply only to that small benchmark and do not establish general accuracy. AddressSanitizer runtime evidence is recorded for CVE-2023-0215 only.

Contents

1 Executive Summary **1**

1.1 Project in one paragraph . . . 1

1.2 Why this research direction was selected . . . 1

1.3 Work completed and evidence . . . 1

1.4 Main conclusion at this stage . . . 2

2 Research Problem and Essential Concepts **3**

2.1 The research question . . . 3

2.2 What is a CVE? . . . 3

2.3 What is typestate? . . . 3

2.4 Important memory-safety terms . . . 4

2.5 Why compare vulnerable and fixed code? . . . 4

2.6 What static source-pattern checking can and cannot do . . . 4

3 How the Project Was Selected **5**

3.1 Stage 1: Select the research theme before selecting individual CVEs . . . 5

3.2 Stage 2: Build a broad candidate register . . . 5

3.3 Stage 3: Apply a five-criterion rubric . . . 6

3.4 The corrected scoring table . . . 7

3.5 Why these three cases won . . . 7

3.6 Why severity alone was not the selection criterion . . . 8

4 Methodology and End-to-End Workflow **9**

4.1 The workflow used . . . 9

4.2 Pipeline overview . . . 10

4.3 Why separate vulnerable and fixed worktrees? . . . 10

4.4 Evidence labels used in this report . . . 10

5 OpenSSL Source Setup and Revision Tracking **12**

5.1 Project layout . . . 12
5.2 Revisions recorded . . . 12
5.3 Files and directories used by the prototype . . . 13

6 Selected CVEs: Root Cause, Fix, and Rule 14
6.1 CVE-2023-0215: BIO-chain use-after-free . . . 14
6.1.1 Where the issue occurs . . . 14
6.1.2 Why the bug is a typestate/ownership error . . . 14
6.1.3 The patch-level idea . . . 14
6.1.4 Rule TS-BIO-01 . . . 15
6.1.5 Runtime validation with AddressSanitizer . . . 15
6.2 CVE-2022-4450: PEM output-pointer double-free risk . . . 15
6.2.1 Where the issue occurs . . . 15
6.2.2 Root cause in plain language . . . 15
6.2.3 The fix and invariant . . . 16
6.2.4 Rule TS-PEM-02 . . . 16
6.2.5 Current validation boundary . . . 16
6.3 CVE-2024-4741: TLS read-buffer lifetime and packet state . . . 16
6.3.1 Where the issue occurs . . . 16
6.3.2 Root cause in plain language . . . 17
6.3.3 Fix and invariant . . . 17
6.3.4 Rule TS-SSL-03 . . . 17
6.3.5 Current validation boundary . . . 17
6.4 What is common across the three cases? . . . 17

7 The Initial Python Checker 19
7.1 What the checker does . . . 19
7.2 Rule summary . . . 19
7.3 How function extraction works . . . 19
7.4 Illustrative pseudo-workflow . . . 20
7.5 Why separate rules are used . . . 20

8 Regression Tests and Runtime Evidence 21
8.1 Combined regression command . . . 21
8.2 Observed source-pattern results . . . 21
8.3 What this result proves . . . 21

8.4 What it does not prove . . . 21
8.5 Syntax checks and individual tests . . . 22

9 Version 2 Evaluation Harness: Results and Critical Reading 23
9.1 Why a separate benchmark was added . . . 23
9.2 The 16 cases . . . 23
9.3 Reported confusion matrix and metrics . . . 24
9.4 Why these metrics must not be overclaimed . . . 24
9.5 How to improve the benchmark . . . 25

10 Recurrence Search: Progress and Evidence Quality 26
10.1 What the recurrence investigation is trying to answer . . . 26
10.2 What the saved notes report . . . 26
10.3 Why the recurrence audit is not yet fully reproducible . . . 26
10.4 What every recurrence record should contain . . . 27
10.5 How to phrase any future candidate . . . 27

11 Formal Rule Models and Current Gaps 28
11.1 TS-BIO-01: chain detachment before freeing . . . 28
11.2 TS-PEM-02: output pointer reset after free . . . 28
11.3 TS-SSL-03: dependent record state consistent with buffer release . . . 29
11.4 Why these models are called candidate models . . . 29

12 Milestone Assessment and Plan 30
12.1 Progress Meeting 1 status . . . 30
12.2 Next milestone dates and deliverables . . . 30
12.3 Recommended work before Progress Meeting 2 . . . 31
12.4 Possible longer-term improvements . . . 31

13 Team Workflow and Reproducibility 32
13.1 Suggested two-member division . . . 32
13.2 Reproducibility commands . . . 32
13.3 What to save after each final run . . . 33

14 Limitations and Threats to Validity 34
14.1 Implementation limitations . . . 34
14.2 Evaluation limitations . . . 34

14.3 Dynamic-test limitations 34
14.4 Recurrence and disclosure limitations 35
14.5 Threats to validity 35

15 Conclusion **36**

A Quick Explanation for the Progress Meeting **37**
A 30-second explanation 37
A 2-minute explanation 37

B Glossary **38**

C Official and Project References **39**

D Final Checklist Before Submission **40**

List of Tables

8.1 Recorded result of the combined source-pattern regression test. 21

9.1 Metrics reported by the Version 2 snippet evaluation harness. 24

Chapter 1

Executive Summary

1.1 Project in one paragraph

The project, **CVE-Guided Typestate Analysis for Recurring Vulnerability Detection in OpenSSL**, investigates whether patterns from known security bugs can be translated into reusable rules about object state, ownership, API contracts, and resource lifetime. OpenSSL is the target codebase. The method begins with public CVEs and their patches, compares vulnerable and fixed revisions, writes a rule for the defect pattern, tests that rule on both versions, and then searches other code locations for similar patterns. A matching pattern is only a candidate for review; it is not automatically a new vulnerability.

1.2 Why this research direction was selected

This project is suitable for a source-based research prototype because OpenSSL has public security advisories, historical source revisions, and fixing commits that can be compared. The selected bugs are not merely three unrelated crashes. They cover three related but distinct lifecycle mistakes: freeing a BIO while chain ownership is inconsistent, leaving caller-visible output pointers referring to freed storage, and freeing a TLS read buffer while dependent record-layer state is stale. These provide concrete examples from which to derive rules.

1.3 Work completed and evidence

Work area	Evidence currently recorded	Status
Project selection	Eight CVE candidates, five-criterion rubric, corrected score table, and rationale for selecting the top three.	Complete for scope-setting
OpenSSL revisions	Separate vulnerable and fixed worktrees for all three selected CVEs, with revisions recorded.	Complete
Checker	`typestate_check.py` contains three CVE-specific checks. It is a heuristic source-pattern checker, not a full typestate engine.	Prototype complete

Regression tests	Combined script reports ALERT for each vulnerable source and NO ALERT for the corresponding fixed source; all three pairs PASS.	Complete for these pairs
Runtime analysis	AddressSanitizer reported a heap-use-after-free on the vulnerable CVE-2023-0215 checkout. The fixed checkout had no corresponding ASan report in the recorded run, though the command still returned an error.	One case tested
Version 2 snippet benchmark	16 hand-written snippets: TP=3, TN=13, FP=0, FN=0; printed precision/recall/specificity of 100%.	Passes limited harness
Recurrence investigation	Notes describe ten candidate locations and broad categories, but important file/function identifiers and source-pinned evidence are missing in the saved audit artefact. No new vulnerability is confirmed.	Needs reconstruction
General accuracy	No independent large-scale source corpus, unbiased precision/recall evaluation, or comprehensive false-positive study.	Future work

1.4 Main conclusion at this stage

The first milestone guideline was to firm up scope, access the target open-source software, and get an initial pipeline running. The project has credible evidence for those three goals. It does *not* yet demonstrate a general-purpose typestate analyzer, general detection accuracy, or successful discovery of a new OpenSSL vulnerability. Those are future objectives.

Chapter 2

Research Problem and Essential Concepts

2.1 The research question

The central research question is:

> Can source-level rules inferred from known OpenSSL vulnerabilities and their fixes detect similar resource-lifecycle or API-state mistakes at other locations in the codebase?

This question contains two separate tasks. The first is to describe the known bug precisely enough to write a check. The second is to determine whether that check remains useful beyond the exact code that inspired it. The first task can be evaluated using vulnerable/fixed revisions. The second needs independent code locations, careful labels, and manual validation.

2.2 What is a CVE?

A Common Vulnerabilities and Exposures (CVE) identifier is a public identifier for a disclosed vulnerability. In this project, a CVE is a starting point for investigation, not a detection rule. The advisory helps identify the affected component and impact; the fixing patch reveals the actual code change; and the vulnerable revision helps explain the pre-fix behavior.

2.3 What is typestate?

Typestate is the idea that the operations permitted on an object depend on its current state. For example, a resource may be allocated, initialized, linked into a data structure, used, detached, and finally freed. A function may be safe only after a required state transition has occurred.

A simplified BIO lifecycle could be written as:

$$\text{Allocated} \rightarrow \text{Linked} \rightarrow \text{Detached} \rightarrow \text{Freed}.$$

If the object is still linked into a chain, freeing it without first restoring the chain can leave references inconsistent. The useful rule is not “never free a BIO”; freeing is normal. The rule is that freeing must respect the object’s ownership and linkage state.

2.4 Important memory-safety terms

Use-after-free (UAF)
Code accesses an object after its storage has been freed. The memory may have been reused or corrupted.

Double-free The same allocation is freed more than once, which can corrupt allocator metadata or lead to other memory-safety failures.

Dangling pointer
A pointer retains an address after the object at that address is no longer valid.

Ownership The component responsible for eventually releasing a resource. APIs often transfer ownership on success but preserve caller ownership on failure.

Failure-path cleanup
Code that releases temporary resources after an operation fails. Error paths are especially important because initialization can be only partially complete.

Invariant A condition that must remain true for the program to be safe. Examples include "no live chain points to a freed BIO" and "a freed output buffer is not exposed through a non-NULL output parameter."

Regression test A repeatable test used to check that a known property remains true after code changes.

AddressSanitizer (ASan)
A compiler/runtime instrumentation tool that can detect errors such as heap use-after-free during an executed program run.

2.5 Why compare vulnerable and fixed code?

A vulnerability description is not always precise enough to implement a reliable check. Comparing the two source revisions helps answer: What changed? Which path was unsafe? What state did the program incorrectly preserve? Which small condition or operation did the patch add? The difference often suggests a candidate invariant.

The vulnerable/fixed comparison is strong evidence for the specific historical defect, but it is not sufficient evidence that a new source location is vulnerable. Recurrence candidates need separate inspection of control flow, ownership, and reachability.

2.6 What static source-pattern checking can and cannot do

The current checker uses simple source-text patterns and rudimentary function-body extraction. It can look for a familiar combination such as a chain push, an error cleanup, and no visible pop. It cannot reliably prove that the pop executes on every path, track arbitrary aliases, understand every macro, or follow ownership transfer across functions. A proper typestate analyzer would need stronger parsing and state/control-flow analysis.

Chapter 3

How the Project Was Selected

3.1 Stage 1: Select the research theme before selecting individual CVEs

The team first chose a research direction rather than starting with one randomly selected vulnerability: use public OpenSSL CVEs to study recurring state, ownership, cleanup, and resource-lifetime errors. This direction has a practical advantage: a public fix supplies a before/after comparison. It also gives a plausible research path from a known bug toward a rule that can be reused at other source locations.

The target was OpenSSL because the project could access its repository and historical revisions, and the public advisory/patch trail supported source investigation. The aim was not to build an exploit or to rank vulnerabilities only by severity. The aim was to learn from patches and test whether the defect could be stated as a useful source-level invariant.

3.2 Stage 2: Build a broad candidate register

The initial register `cve_candidates_openssl.csv` listed eight candidates. A register broader than the final selection is useful because it makes the selection decision visible and lowers the risk of choosing only cases that already fit the intended approach. Early descriptions, function names, and version ranges were treated as screening metadata; the exact source and fix had to be verified before use.

Candidate	Reported issue	Potential rule family	Screening view
CVE-2023-0215	UAF associated with BIO_new_NDEF and BIO-chain cleanup.	Chain ownership, cleanup after failure, object lifetime.	Strong fit
CVE-2022-4450	Double-free risk after PEM_read_bio_ex failure.	Output-pointer validity and ownership on failure.	Strong fit
CVE-2024-4741	UAF associated with SSL buffer release.	Buffer lifetime and dependent packet state.	Strong fit
CVE-2023-5363	Incorrect processing of cipher key and IV lengths.	Parameter validation / API configuration order.	Possible, but less clearly typestate

Continued on the next page

Candidate	Reported issue	Potential rule family	Screening view
CVE-2023-0401	NULL dereference during PKCS7 verification.	Object-validity guard.	A missing NULL check alone may not form a reusable state rule
CVE-2023-0216	Invalid pointer dereference in d2i_PKCS7 family.	Pointer validity and decoding state.	Possibly specific to a parser path
CVE-2026-45447	Heap UAF reported in PKCS7_verify.	Object lifetime and verification cleanup.	Provisional; not selected for initial prototype
CVE-2026-35188	Double-free reported in OCSP stapled response handling.	Response ownership and cleanup.	Provisional; not selected for initial prototype

The two 2026 cases were included as later candidates, but the project did not use them in the prototype. OpenSSL's current vulnerability pages identify CVE-2026-45447 as a high-severity heap UAF in PKCS7_verify and CVE-2026-35188 as a double-free during OCSP stapled-response checking. Their inclusion in the register does not mean their patches or source-level recurrence rules were implemented in this project. See Appendix C for source links.

3.3 Stage 3: Apply a five-criterion rubric

Each candidate received a score from 0 to 2 on five criteria. The maximum is 10. These are prioritization judgments rather than measured detection outcomes.

Criterion	What a score measures	Scoring guide
Patch/source availability	Is there enough public material to compare versions?	0 = not established; 1 = partly located or needs verification; 2 = source and relevant fix appear available.
Root-cause clarity	Can the underlying error be explained from code and patch?	0 = unclear; 1 = partly understood; 2 = plausible, explainable source-level cause.
Typestate/API rule fit	Can the defect be expressed as an invariant or invalid state transition?	0 = weak fit; 1 = possible but uncertain; 2 = strong fit to state, ownership, cleanup, or API contract.

Continued on the next page

Criterion	What a score measures	Scoring guide
Recurrence potential	Could a similar mistake occur at another source location?	0 = likely isolated; 1 = possible; 2 = plausible reusable pattern.
Time feasibility	Can the case be investigated within the first milestone?	0 = infeasible; 1 = uncertain/significant effort; 2 = manageable.

3.4 The corrected scoring table

CVE	Patch	Cause	Rule fit	Rec.	Time	Total / decision
CVE-2023-0215	2	2	2	2	2	**10/10 – selected**
CVE-2022-4450	2	2	2	2	2	**10/10 – selected**
CVE-2024-4741	2	2	2	1	2	**9/10 – selected**
CVE-2023-5363	2	2	1	1	2	8/10 – omitted
CVE-2023-0401	2	2	1	1	2	8/10 – omitted
CVE-2023-0216	2	2	1	1	2	8/10 – omitted
CVE-2026-45447	1	2	2	1	1	7/10 – provisional, omitted
CVE-2026-35188	1	1	2	1	1	6/10 – provisional, omitted

Score reconciliation. An earlier copy of the scoring table recorded a total of 7 for CVE-2023-0216 even though its five component scores add to 8. The Version 2 register reconciled that arithmetic and records 8/10. The correction does not change the three selected cases, which remain the highest-scoring three.

3.5 Why these three cases won

- **CVE-2023-0215**: a clean resource-lifecycle and failure-cleanup example. The patch explicitly restores BIO-chain state before freeing the filter. It was also feasible to investigate dynamically with AddressSanitizer.

- **CVE-2022-4450**: a complementary ownership example. It concerns pointer values exposed through output parameters after memory has been freed. The invariant differs from BIO chain membership, so it motivates a separate rule.
- **CVE-2024-4741**: a third lifecycle example in the TLS record layer. The fix synchronizes dependent packet-pointer and length fields with buffer release and guards against release while data remains in use.

3.6 Why severity alone was not the selection criterion

Security severity matters for prioritizing risk, but it does not answer whether a bug is a good fit for this particular research method. A high-severity issue can require complex path reasoning or parameter-specific logic and therefore be difficult to turn into a first source-pattern rule. The rubric prioritizes a combination of traceable patch evidence, explainable root cause, state/API relevance, plausible recurrence, and time feasibility.

Chapter 4

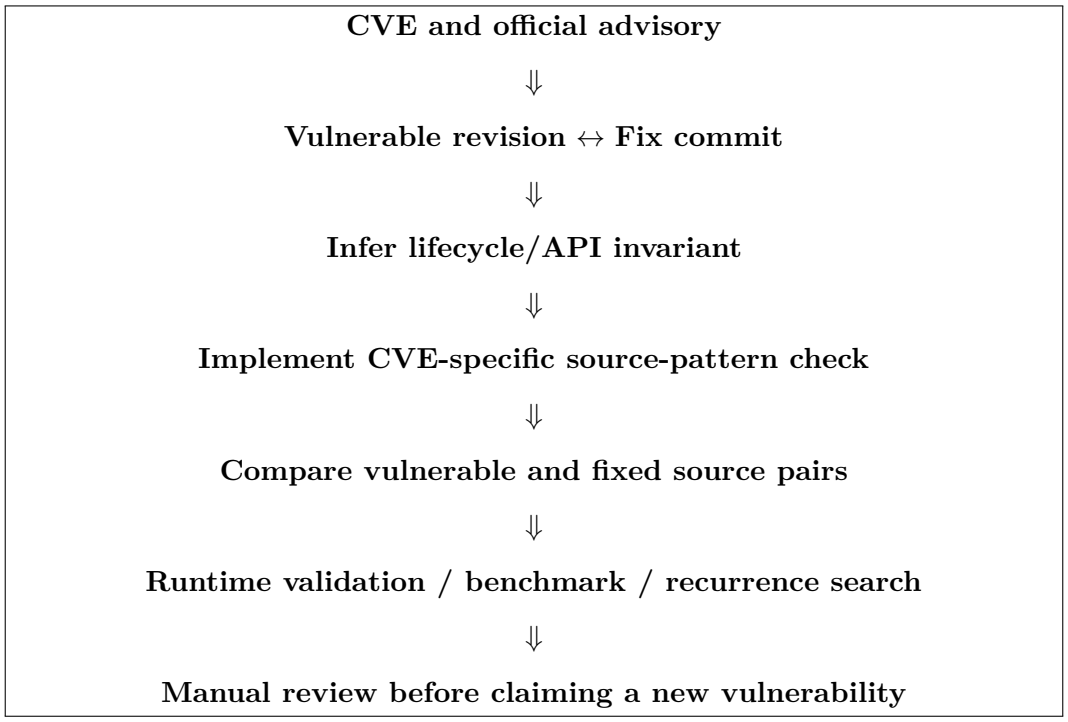
Methodology and End-to-End Workflow

4.1 The workflow used

The investigation follows this sequence:

1. Find candidate CVEs and official advisory records.
2. Record them in the candidate CSV and mark uncertainty rather than guessing.
3. Score them using the five-criterion rubric.
4. Choose the three highest-scoring cases for the first prototype.
5. Check out vulnerable revisions and corresponding fix commits.
6. Read the patch and identify the unsafe operation, affected object(s), and corrected invariant.
7. Encode a candidate rule in the Python checker.
8. Run the source checker against both vulnerable and fixed source files.
9. Run regression scripts to make the source-pattern comparison repeatable.
10. Where feasible, execute a runtime test under AddressSanitizer.
11. Search additional locations for matching patterns, recording the evidence and uncertainty.
12. Report results with clear boundaries between source evidence, test evidence, and claims of generalization.

4.2 Pipeline overview



4.3 Why separate vulnerable and fixed worktrees?

Keeping both revisions available reduces the chance of accidentally testing the same source twice. It also makes the comparison repeatable: the checker receives a path into a known vulnerable tree and the corresponding path into the fixed tree. The exact revision matters because behavior, source layout, and helper functions can change between branches.

4.4 Evidence labels used in this report

Advisory-verified An official public record supports the broad vulnerability description and release-fix context.

Source/patch-verified
The project notes record a relevant source file, revision, and code change.

Regression-tested The checker distinguished the selected vulnerable/fixed source pair.

Runtime-tested A program was executed and a runtime detector produced a memory-safety result.

Snippet-benchmark-tested
A separate harness evaluated hand-written miniature code strings rather than loading full source files from the OpenSSL tree.

Recurrence candidate
A location worth investigating. It is not a confirmed vulnerability until its

code path, ownership, and consequences are manually validated.

Chapter 5

OpenSSL Source Setup and Revision Tracking

5.1 Project layout

The reported working root in WSL is:

```
/mnt/d/CVE-Typestate-Project/
```

The repository setup includes a main checkout, `openssl/`, and six case-specific trees:

```
openssl-cve-2023-0215-vulnerable/
openssl-cve-2023-0215-fixed/
openssl-cve-2022-4450-vulnerable/
openssl-cve-2022-4450-fixed/
openssl-cve-2024-4741-vulnerable/
openssl-cve-2024-4741-fixed/
```

5.2 Revisions recorded

CVE	Vulnerable revision / parent	Fix commit
CVE-2023-0215	cbafa34b5a057794c5c0 8cd4657038e1f643c1ac	8818064ce3c3c0f1b740 a5aaba2a987e75bfbafd
CVE-2022-4450	8e257b86e5812c6e1cfa 9e8e5f5660ac7bed899d	63bcf189be73a9cc1264 059bed6f57974be74a83
CVE-2024-4741	b3f0eb0a295f58f16ba4 3ba99dad70d4ee5c437d	2d05959073c4bf880340 1668b9df85931a08e020

These identifiers are reproduced from the project's recorded worktree notes. The vulnerable revision is described as the parent of the relevant fix commit used in the historical comparison. Before publishing a public repository, retain a machine-readable record of each worktree's actual 'git rev-parse HEAD' output so that the final repository itself independently proves these checkouts.

5.3 Files and directories used by the prototype

Path / file	Purpose
`typestate_check.py`	Main Python source-pattern checker with three CVE-specific rules.
`tests/test_cve_2023_0215.sh`	Individual vulnerable/fixed regression test for the BIO-chain case.
`tests/test_cve_2022_4450.sh`	Individual vulnerable/fixed regression test for PEM output pointers.
`tests/test_all_cves.sh`	Runs all three source-pattern regression pairs.
`CVE-2023-0215-asan-result.txt`	Notes about the vulnerable AddressSanitizer run.
`CVE-2023-0215-fixed-result.txt`	Notes about the fixed-checkout runtime result.
`CVE-2023-0215-evidence.txt`	Source evidence and revision details for CVE-2023-0215.
`CVE-2022-4450-evidence.txt`	Source evidence and detection outcome for CVE-2022-4450.
`CVE-2024-4741-evidence.txt`	Source evidence and detection outcome for CVE-2024-4741.
`README.md`	Project introduction, file inventory, running instructions, and limitations.
`typestate_rules_spec_v2.md`	Written state models and rule hypotheses for TS-BIO-01, TS-PEM-02, and TS-SSL-03.
`typestate_checker_eval_v2.py`	Separate 16-case hand-written snippet benchmark.
`eval_v2_stdout.log`	Recorded output from the 16-case snippet harness.
`recurrence_investigation_v2.md`	Notes describing ten recurrence-search locations and classifications; saved identifiers are incomplete and need repair.

Chapter 6

Selected CVEs: Root Cause, Fix, and Rule

6.1 CVE-2023-0215: BIO-chain use-after-free

6.1.1 Where the issue occurs

The principal source file is `crypto/asn1/bio_ndef.c`, in `BIO_new_NDEF()`. A BIO is an OpenSSL I/O abstraction; BIOs can be linked into chains so data can pass through multiple filters. In the relevant setup, an ASN.1 filter BIO is pushed into a chain. If initialization fails and cleanup releases the filter without restoring the expected chain structure, the chain can be left in an invalid state.

6.1.2 Why the bug is a typestate/ownership error

The issue is not simply that memory is freed. A filter BIO can have relationships with neighbouring BIOs and chain pointers. Its lifetime and those relationships must remain consistent. The fix commit documents the ownership contract: on success, the returned BIO owns the input BIO as part of its BIO chain; on failure, NULL is returned and the input BIO remains owned by the caller. That contract makes the failure path important.

6.1.3 The patch-level idea

`8818064ce3c3c0f1b740`
The recorded fix commit is `a5aaba2a987e75bfbafd`. It adds tracking for the BIO to pop and performs a `BIO_pop()` in the error cleanup before freeing the ASN.1 BIO. Conceptually, the sequence becomes:

1. Create and link the filter BIO.
2. A later setup/callback operation fails.
3. Restore the chain using the appropriate pop operation.
4. Free the temporary BIO and other temporary state.
5. Return failure while leaving the caller's input ownership consistent with the API contract.

6.1.4 Rule TS-BIO-01

Tracked objects: filter BIO and BIO chain.
Intended invariant: if an error path frees a filter that has been inserted into a caller-owned chain, it must first restore the chain relationship, for example by popping the filter.
Forbidden transition:

$$\text{Linked into chain} \rightarrow \text{Freed} \quad \text{without a chain-restoration step.}$$

The checker function `check_bio_ndef()` looks for the relevant chain insertion and error-cleanup pattern, then checks whether the expected pop operation is visible.

6.1.5 Runtime validation with AddressSanitizer

The recorded command was run in the vulnerable OpenSSL checkout:

```
./apps/openssl cms -encrypt -in test/smcont.txt -stream -recip test/smime-certs/smec1
    .pem -des
```

The recorded ASan report was a heap-use-after-free. The use site was `BIO_pop()` in `crypto/bio/bio_lib.c` around line 678; the free was associated with `BIO_new_NDEF()` in `crypto/asn1/bio_ndef.c` around line 100. The trigger was CMS streaming encryption with the selected cipher option, which made cipher initialization fail and exercised the vulnerable cleanup path.

The same command was run against the fixed checkout and did not produce the corresponding ASan memory-safety report. However, the command still returned an error. The appropriate conclusion is therefore limited: the memory-safety failure reproduced in the recorded vulnerable run and was not reported in that fixed run. This is useful runtime evidence for this case, not proof that every path in the fixed code is safe.

6.2 CVE-2022-4450: PEM output-pointer double-free risk

6.2.1 Where the issue occurs

The principal source file is `crypto/pem/pem_lib.c`, function `PEM_read_bio_ex()`. The function reads a PEM object and provides results through output parameters, including header and data pointers. The caller is responsible for freeing the returned buffers when ownership has been passed to it.

6.2.2 Root cause in plain language

During a failure path, memory could be freed while a caller-visible output parameter still retained the old address. A later cleanup by a caller could attempt to free that same allocation again. The unsafe state is a pointer that looks like it refers to a valid object even though the allocation has already been released.

6.2.3 The fix and invariant

63bcf189be73a9cc1264
The recorded fixing commit is 059bed6f57974be74a83. The relevant cleanup resets the output parameters after freeing their buffers:

```
out_free:
    pem_free(*header, flags, 0);
    *header = NULL;
    pem_free(*data, flags, 0);
    *data = NULL;
```

This aligns the externally visible pointer state with the allocation lifetime.

6.2.4 Rule TS-PEM-02

Tracked objects: output parameters `*header` and `*data`.
Intended invariant: after freeing a buffer on a failure path, the corresponding caller-visible output pointer is cleared before returning failure.
Forbidden transition:

$$\text{Buffer freed} \rightarrow \text{Return with output pointer still dangling.}$$

The checker function `check_pem_read()` checks the failure cleanup for freed header/data buffers and expected pointer resets.

6.2.5 Current validation boundary

The vulnerable source was reported as ALERT and the fixed source as NO ALERT in the combined source-pattern regression test. The 16-case snippet benchmark also contains paired illustrative cases. A separate AddressSanitizer runtime reproduction was not recorded for this CVE in Progress Meeting 1. The current evidence is source-pattern regression evidence, not a dynamic demonstration of the double-free.

6.3 CVE-2024-4741: TLS read-buffer lifetime and packet state

6.3.1 Where the issue occurs

The principal source is `ssl/record/ssl3_buffer.c`, function `ssl3_release_read_buffer()`, alongside related record-layer logic. The official OpenSSL advisory describes a use-after-free associated with `SSL_free_buffers()` and notes that applications directly calling that API are the affected use case. In some situations, the record-layer buffer could still be in use when it was released.

6.3.2 Root cause in plain language

The allocated read buffer may have other state that points into it. OpenSSL's record layer maintains a packet pointer and packet length. Releasing the buffer while the record-layer state still describes data inside that buffer can leave a stale pointer and inconsistent state. This is a dependency problem: the pointer and length are meaningful only while the backing buffer remains valid and the state is consistent.

6.3.3 Fix and invariant

`2d05959073c4bf880340`

The recorded fixing commit is `1668b9df85931a08e020`. The fixed source resets the packet pointer and length after release, and related code checks record-layer state before releasing a buffer. The state model is:

Buffer allocated → Packet state may reference buffer

→ Confirm safe-to-release state → Reset dependent state → Free buffer.

The exact ordering and guards matter; a text occurrence of a reset is not by itself proof that every path is safe.

6.3.4 Rule TS-SSL-03

Tracked objects: read buffer `b->buf`; record-layer packet pointer `s->rlayer.packet`; and packet length `s->rlayer.packet_length`.
Intended invariant: a buffer must not be released while relevant record data is still active; after release, dependent packet state must not continue to describe memory that is gone.
Forbidden transition:

Record layer depends on buffer → Buffer freed

without the required guard and dependent-state updates.

6.3.5 Current validation boundary

The vulnerable source was reported as ALERT and the fixed source as NO ALERT in the combined source-pattern regression test. Runtime validation was not recorded for this case in Part 1. The snippet benchmark's simplified rule checks whether the packet pointer is reset, but its benchmark logic does not fully establish that both the pointer and packet length are correctly handled on every path; this limitation is described again in Chapter 10.

6.4 What is common across the three cases?

All three involve a relationship between resource lifetime and program-visible state. However, the relationship differs: BIO chain membership; output pointers exposed to callers; and a buffer plus dependent record-layer state. It is not safe to collapse them into a rule such as "a free without a NULL assignment is a bug." A future generalized model could share abstract concepts such

as allocated, linked/referenced, released, and state-consistent, but each API still needs its own contract and validation logic.

Chapter 7

The Initial Python Checker

7.1 What the checker does

The main checker is `typestate_check.py`. At a high level, it reads a source file, attempts to find a relevant function body, runs the corresponding CVE-specific pattern check, and emits a result such as ALERT or NO ALERT. It is a first prototype intended to make the method executable and testable.

7.2 Rule summary

Rule	Object / relation	Pattern hypothesis	Main weakness
TS-BIO-01	BIO filter and chain ownership	A filter is linked/pushed, then freed on a cleanup path without the expected chain pop/restoration.	Textual ‘BIO_pop‘ presence does not prove it runs on all error paths.
TS-PEM-02	‘*header‘ and ‘*data‘ output pointers	A buffer is freed in failure cleanup but the corresponding externally visible output pointer remains non-NULL.	Needs to check each pointer and each exit path, not merely any NULL reset.
TS-SSL-03	Read buffer and record-layer packet state	Read buffer is released while packet pointer/length remain inconsistent or active.	Distributed state and conditions need path-sensitive analysis.

7.3 How function extraction works

The current implementation uses simple brace counting to estimate a function body. This is not a complete C parser. Braces in comments, strings, macros, or preprocessor branches can confuse a simple extractor. The checker also does not comprehensively reason about function calls, aliases, or control-flow paths.

7.4 Illustrative pseudo-workflow

```
read source file
locate function of interest using a simple signature/body search
extract a candidate body using brace counting
apply the CVE-specific string-pattern rule
print ALERT / NO ALERT / SKIP or an error
return a status code for the shell regression script
```

This design is appropriate as a first milestone prototype because it exercises the complete workflow from a supplied source path to a repeatable result. It is not adequate by itself for a claim that the tool can reliably discover previously unknown vulnerabilities across an entire C codebase.

7.5 Why separate rules are used

The three selected CVEs do not share one identical invalid operation. A valid BIO free may still be unsafe if the BIO remains in a linked chain. A valid buffer free may be unsafe if output pointers are not cleared. A buffer release may be unsafe if a dependent TLS packet pointer remains active. Separate rules let the prototype encode each invariant clearly and avoid a single over-broad “free is suspicious” heuristic.

Chapter 8

Regression Tests and Runtime Evidence

8.1 Combined regression command

The reported combined test is run from the project root:

```
cd /mnt/d/CVE-Typestate-Project
bash tests/test_all_cves.sh
```

The regression scripts execute the source checker on the known vulnerable and fixed paths. In the observed run, all three source pairs were distinguished as expected.

8.2 Observed source-pattern results

CVE	Vulnerable source	Fixed source	Pair result
CVE-2023-0215	ALERT	NO ALERT	PASS
CVE-2022-4450	ALERT	NO ALERT	PASS
CVE-2024-4741	ALERT	NO ALERT	PASS

Table 8.1: Recorded result of the combined source-pattern regression test.

The script concluded: `PASS: all three vulnerable/fixed pairs were distinguished.`

8.3 What this result proves

It demonstrates that the current checker can be executed against the selected source paths and that the coded rules recognize the expected pattern differences in those three historical pairs. It also creates a regression test for future edits: a change that makes a rule stop distinguishing its source pair should be caught by the script.

8.4 What it does not prove

The test does not show how often the checker flags unrelated code, how often it misses a vulnerable pattern expressed differently, or whether it finds a new vulnerability. The vulnerable/fixed

examples were used to create the rules, so the regression test is best understood as a consistency check on known examples rather than an independent accuracy evaluation.

8.5 Syntax checks and individual tests

The report records these checks:

```
python3 -m py_compile /mnt/d/CVE-Typestate-Project/typestate_check.py
bash -n /mnt/d/CVE-Typestate-Project/tests/test_all_cves.sh
bash /mnt/d/CVE-Typestate-Project/tests/test_cve_2023_0215.sh
bash /mnt/d/CVE-Typestate-Project/tests/test_cve_2022_4450.sh
bash /mnt/d/CVE-Typestate-Project/tests/test_all_cves.sh
```

The Python compilation check validates syntax only; `bash -n` validates shell syntax only. Neither proves the analyzer's semantic correctness. The actual regression run provides the behavioral result for the tested pairs.

Chapter 9

Version 2 Evaluation Harness: Results and Critical Reading

9.1 Why a separate benchmark was added

The Version 2 package adds `typestate_checker_eval_v2.py`, which runs small, labelled code snippets through simplified versions of the three rules. It was intended to broaden the cases beyond the three positive examples by adding fixed snippets, safe controls, and controls that are not intended to be typestate cases.

9.2 The 16 cases

The harness lists 16 embedded, hand-written code strings:

- 3 positive vulnerable examples: one each for CVE-2023-0215, CVE-2022-4450, and CVE-2024-4741;
- 3 negative fixed examples: one fixed example for each selected CVE;
- 6 safe-control examples;
- 2 additional safe-control examples;
- 2 non-typestate control examples.

The expected labels total 3 alerts and 13 no-alerts. The recorded log reports all 16 labels matched the harness's observed outputs.

Important distinction: these are hand-written illustrative snippets embedded in the Python evaluation file. The harness does not load each real C function from the OpenSSL worktrees and execute the full production checker on the actual source. The filenames and function names in the test descriptions identify the intended analogue, but they do not make the embedded strings identical to source-pinned repository functions. Therefore, this is a small unit-style demonstration of the substring rules, not an independent evaluation over 16 real OpenSSL functions.

9.3 Reported confusion matrix and metrics

Measure	Value	Interpretation
True positives (TP)	3	Three labelled vulnerable snippets flagged
True negatives (TN)	13	Thirteen labelled non-alert snippets not flagged
False positives (FP)	0	No labelled negative snippet was flagged
False negatives (FN)	0	No labelled positive snippet was missed
Total cases	16	Small hand-written suite
Precision	100%	TP/(TP+FP) for this suite
Recall	100%	TP/(TP+FN) for this suite
Specificity	100%	TN/(TN+FP) for this suite
F1 score	100%	Harmonic mean of precision and recall for this suite

Table 9.1: Metrics reported by the Version 2 snippet evaluation harness.

The formulas are:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN},$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \qquad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

With the logged values TP=3, TN=13, FP=0, and FN=0, the formulas produce 100% on this particular test set.

9.4 Why these metrics must not be overclaimed

The benchmark is very small, manually authored, and closely coupled to the rules that it tests. It therefore provides little evidence about performance on unseen code. A source matcher can pass a suite whose positive examples contain the expected substrings and whose negative examples omit them, yet still fail on realistic C code with macros, multiple branches, aliases, helper functions, comments, or equivalent code written differently.

There are also concrete gaps between the rule specification and the benchmark implementation:

- The PEM snippet rule treats the presence of a reset to `*header`, `*data`, or a generic pointer as sufficient to avoid an alert. That does not prove that *both* output pointers are reset after their respective frees. A test containing only one reset could therefore be incorrectly accepted by the benchmark logic.
- The simplified SSL snippet rule checks for `s->rlayer.packet = NULL`, but does not fully enforce the specification that both the packet pointer and packet length are reset correctly on every relevant path.
- The BIO snippet logic checks whether a pop token appears anywhere in the embedded code; it does not prove the pop occurs on every failure path or before the free on that same path.

For these reasons the technically accurate statement is: **the Version 2 hand-written snippet harness reported 100% precision, recall, specificity, and F1 on its 16 examples; this**

does not establish 100% detection accuracy for the real checker or for OpenSSL source generally.

9.5 How to improve the benchmark

The next benchmark should load source-pinned vulnerable and fixed functions from known commits. Negative controls should be selected independently of the rule design and cover realistic near-misses, including conditional pops, helper functions, macros, alternate names, one missing pointer reset, and stale packet length despite a cleared pointer. The labels should be reviewed by a person other than the rule author where feasible. Every case should record source path, commit SHA, line range, expected result, observed result, and rationale.

Chapter 10

Recurrence Search: Progress and Evidence Quality

10.1 What the recurrence investigation is trying to answer

Regression on a known CVE asks whether the rule recognizes the original defect and its fix. Recurrence analysis asks a harder question: does a similar unsafe state transition occur elsewhere? That step is essential to the project's long-term research goal, but it is also where false positives become dangerous. Similar syntax does not necessarily imply the same ownership contract or reachable failure mode.

10.2 What the saved notes report

The Version 2 recurrence notes describe ten candidate locations across OpenSSL modules. Their summary classification is:

- eight locations described as safe controls or cases that adhere to the invariant;
- one candidate rejected as out of scope because the relevant BIO reference was standalone rather than part of the caller-owned chain assumed by the rule;
- one unresolved candidate involving a complex internal context state machine that the current heuristic could not resolve.

The notes do not report a confirmed new vulnerability.

10.3 Why the recurrence audit is not yet fully reproducible

The saved recurrence Markdown file contains missing file names, function identifiers, and other fields in the audit table. It gives high-level explanations, but important details needed to reproduce each conclusion are absent. This means the ten-location summary can be retained as a record of exploratory progress, but it should not be presented as a complete, source-pinned audit trail without repairing the missing evidence.

10.4 What every recurrence record should contain

For each candidate, capture:

1. a unique candidate ID;
2. exact repository path and function name;
3. commit SHA and line range;
4. the relevant source excerpt, with enough surrounding code to understand the operation;
5. which rule triggered and what exact sequence matched;
6. the ownership/API contract that makes the case analogous to the original CVE;
7. the call-path or error-path reasoning;
8. a classification: safe control, rejected/inapplicable, unresolved, or potential bug;
9. a reviewer note and the next validation step.

10.5 How to phrase any future candidate

Before validation, write "potential recurrence candidate requiring manual review," not "new vulnerability found." A confirmed report needs a trace from the matched source pattern to a reachable execution path and an explanation of the actual consequence. Where relevant, runtime reproduction, sanitizer evidence, or a minimal test can strengthen the analysis. Any upstream report should follow the project's responsible-disclosure expectations.

Chapter 11

Formal Rule Models and Current Gaps

11.1 TS-BIO-01: chain detachment before freeing

Element	Current model
Tracked entity	A filter BIO and the BIO chain into which it is pushed.
Abstract states	Allocated; linked into chain; setup/callback failed; detached/restored; freed.
Allowed safe cleanup	Linked into chain → restore/detach chain → free temporary BIO.
Forbidden transition	Linked into chain → free without restoring relevant links.
Observable heuristic	Chain push / BIO creation, an error cleanup with free, and absence of a visible pop.
Main unmodelled details	Aliasing, helper calls, conditional reachability, chain mutation by callbacks, and ownership transfer across functions.

11.2 TS-PEM-02: output pointer reset after free

Element	Current model
Tracked entity	Output parameters `*header` and `*data` and the buffers they point to.
Abstract states	Empty; allocated internally; returned to caller; freed on failure; output pointer cleared.
Allowed safe failure cleanup	Free each owned buffer and set its corresponding output pointer to NULL before returning failure.
Forbidden transition	Free buffer → return with the corresponding output pointer still dangling.
Observable heuristic	Free of header/data buffer plus missing pointer reset.
Main unmodelled details	Whether output parameters are guaranteed initialized, API error contract, multiple exits, ownership transfer, aliases, and macro-expanded frees.

11.3 TS-SSL-03: dependent record state consistent with buffer release

Element	Current model
Tracked entity	SSL read buffer and dependent fields `s->rlayer.packet` and `s->rlayer.packet_length`.
Abstract states	Buffer unallocated; buffer allocated; record layer may point into buffer; release is permitted; dependent state reset; buffer freed.
Allowed safe release	Establish release preconditions, clear/reset dependent state as required, and release only when no active record data depends on the buffer.
Forbidden transition	Free buffer while dependent state can still refer to it or while the release guard is not satisfied.
Observable heuristic	Detect buffer free and look for required packet state updates.
Main unmodelled details	Cross-function state changes, exact path conditions, partial record state, aliasing, and the semantics of helper macros/functions.

11.4 Why these models are called candidate models

The models articulate the intended invariant clearly enough to guide implementation, but they are not yet complete formal semantics of the OpenSSL APIs. For example, the TLS rule involves a guard and distributed state, while the compact benchmark only checks selected substrings. A full analyzer would need to prove that the required conditions hold across every relevant path, not just detect a text pattern.

Chapter 12

Milestone Assessment and Plan

12.1 Progress Meeting 1 status

The stated first-meeting guideline was: (1) scope firmed up, (2) target open-source software accessible, and (3) initial pipeline running.

Milestone expectation	Evidence	Assessment
Scope firmed up	Eight candidates were registered and scored. Three selected CVEs now have separate rule hypotheses and source worktrees.	Complete
Target OSS accessible	OpenSSL main checkout and vulnerable/fixed worktrees are present in the recorded project layout.	Complete
Initial pipeline running	Python checker and combined regression script distinguish all three selected vulnerable/fixed pairs.	Complete
Runtime testing for all three CVEs	ASan record exists for CVE-2023-0215 only.	Partial
Recurrence evidence	Ten locations are described in notes, but missing identifiers/source excerpts prevent full independent reproduction.	Needs work
General detection accuracy	No independent, representative real-source benchmark.	Future work
Confirmed new vulnerability	None confirmed in the notes.	Not claimed

12.2 Next milestone dates and deliverables

The working schedule recorded for the project is:

- **Progress Meeting 1: 11–16 October 2026.** Scope, accessible target OSS, and initial pipeline. This report captures the current evidence.
- **Progress Meeting 2: 26–30 October 2026.** Aim for first end-to-end results, first bug

candidates if justified by evidence, and an evaluation plan.

- **Progress Meeting 3: 2–6 November 2026.** Aim to consolidate results, draft the report, and prepare any justified upstream reports/PRs. Filing an upstream report is a planned activity, not a completed result at the date of this document.
- **Final presentation: Thursday, 12 November 2026.** Expected format: 10–15 minutes per group plus Q&A in the scheduled Teams session.
- **Paper and repository due: Friday, 13 November 2026, 23:59 IST.** Deliver the Overleaf project link, PDF, public GitHub repository, and reproducibility instructions via Teams.

12.3 Recommended work before Progress Meeting 2

1. Repair the recurrence audit table: restore exact paths/functions, source lines, commit identifiers, and evidence snippets for all ten locations.
2. Make the benchmark read real source functions from fixed commits rather than using embedded illustrative snippets.
3. Add adversarial near-miss tests: one output pointer reset but not the other; a pop operation on a different branch; packet pointer reset but packet length stale; macros or helper functions hiding a release.
4. Re-run and save the combined test, Python compilation check, shell syntax check, and benchmark output from the final working tree. Include timestamps and commit IDs.
5. Consider dynamic reproductions for CVE-2022-4450 and CVE-2024-4741 if time and environment permit, without making them a blocker for the existing milestone unless required by the supervisor.
6. Keep source-level and runtime results in separate columns in every summary table.

12.4 Possible longer-term improvements

The current approach can be strengthened in stages. First, harden the function extraction and add precise path-specific checks. Second, use a C parser such as Clang tooling or a Tree-sitter-based parser to avoid fragile brace matching. Third, add control-flow/data-flow reasoning for the state variables and paths. Fourth, evaluate on independently selected source functions, including safe controls that resemble the bug syntactically. Finally, perform manual review before treating any recurrence candidate as a bug.

Chapter 13

Team Workflow and Reproducibility

13.1 Suggested two-member division

The following is a proposed division for a two-member project, not a claim that named assignments were already formally recorded.

Work package	Member A: research/source lead	Member B: tooling/evaluation lead
Candidate discovery	Collect advisories, fill candidate register, record source uncertainty.	Check source links, patch identity, version metadata, and feasibility.
Rubric	Explain component scores and selection rationale.	Audit totals and challenge inconsistent scoring.
CVE analysis	Explain root cause, source diff, and invariant.	Independently verify patch and testability.
Rules	Write the state model and expected violation.	Implement the rule and edge cases.
Testing	Interpret what the output does and does not show.	Run source pairs, syntax checks, benchmark, and dynamic tests where feasible.
Recurrence audit	Explain why a candidate matches the invariant.	Record path, lines, commit, test status, and safe/rejected/unresolved classification.
Report	Draft technical rationale and limitations.	Maintain commands, logs, reproducibility steps, and test tables.
Cross-review	Review rule code and scripts.	Review CVE interpretation and evidence boundaries.

13.2 Reproducibility commands

The following commands are recorded as the basic checks. They assume the stated project root exists in the WSL environment.

```
cd /mnt/d/CVE-Typestate-Project
```

```
# Python syntax check
python3 -m py_compile typestate_check.py

# Shell syntax check only (does not execute the tests)
bash -n tests/test_all_cves.sh

# Run the recorded regression tests
bash tests/test_cve_2023_0215.sh
bash tests/test_cve_2022_4450.sh
bash tests/test_all_cves.sh
```

To reproduce the Version 2 miniature-snippet benchmark, run:

```
python3 /path/to/typestate_checker_eval_v2.py
```

Use the actual location of that file inside the project/package. The benchmark prints its labels, observed outputs, confusion matrix, and caveat.

13.3 What to save after each final run

Record the exact command, current Git commit, date/time, standard output, standard error if any, return code, and relevant source path. If a run fails, preserve the failure rather than replacing it with a success-only screenshot. A future reviewer should be able to distinguish a tool crash, a no-alert result, a test failure, and a runtime error.

Chapter 14

Limitations and Threats to Validity

14.1 Implementation limitations

- The main checker is based on source-pattern heuristics and simple brace counting, not a complete parser or formal typestate engine.
- String matching does not establish the order or reachability of operations across all control-flow paths.
- The prototype does not comprehensively model aliasing, interprocedural ownership transfers, callback side effects, macros, or all compiler/preprocessor cases.
- The three rules are CVE-specific and were designed around the historical examples. Their success on those cases cannot be treated as an independent test of generalization.

14.2 Evaluation limitations

- The 16-case evaluation file embeds hand-written snippets rather than loading 16 independent real OpenSSL functions.
- Positive examples are closely coupled to the tokens expected by the rule, which makes perfect scores less informative than a truly independent test set.
- The PEM benchmark logic accepts the presence of a reset to either header or data as sufficient for the simplified no-alert decision; it does not require both resets.
- The SSL benchmark checks packet-pointer reset but does not fully test packet-length reset and path conditions.
- No large independent benchmark, class-balanced external corpus, or formal false-positive/false-negative study is available yet.

14.3 Dynamic-test limitations

The recorded AddressSanitizer experiment covers CVE-2023-0215 only. A fixed-checkout run without an ASan memory-safety report is reassuring for that execution, but the command returned an error and no universal claim about the fixed implementation should be made from one test. Runtime reproductions for CVE-2022-4450 and CVE-2024-4741 were not recorded in Part 1.

14.4 Recurrence and disclosure limitations

The saved recurrence audit has missing identifiers and lacks sufficient source-pinned excerpts for each row. No new vulnerability has been confirmed. A pattern match is evidence for manual review, not proof of exploitability or a basis for claiming a discovery. Any later disclosure should include a reproducible technical description and follow responsible-disclosure guidance.

14.5 Threats to validity

Construct validity
: The source patterns may not fully capture the intended typestate invariant. Token presence is only an approximation of state-transition correctness.

Internal validity
: The rules and benchmark snippets were authored together, so the benchmark may favour the implementation.

External validity
: Three CVEs from one library and 16 hand-written snippets are not representative of all C libraries or OpenSSL functions.

Reproducibility validity
: Missing source identifiers in the recurrence artefact and reliance on recorded outputs make clean regeneration especially important.

Chapter 15

Conclusion

The project selection followed a defensible sequence: choose a research problem centred on resource lifecycle and API state, discover eight candidate CVEs, score them against a transparent five-part rubric, and select three cases that combine clear patch evidence with useful ownership/lifetime patterns. The scoring record for CVE-2023-0216 was reconciled from 7 to 8 without changing the selected top three.

For the three selected CVEs, the project records vulnerable/fixed OpenSSL worktrees, patch-derived rule hypotheses, a Python source-pattern checker, and shell regression tests. The combined test distinguished all three vulnerable/fixed pairs. CVE-2023-0215 additionally has AddressSanitizer evidence on the vulnerable checkout and a fixed-checkout run without the same memory-safety report.

The newer Version 2 benchmark reports perfect metrics across 16 hand-written snippets. This result should be reported with its scope: it shows that the miniature tests and simplified string rules agreed on their labels; it does not measure broad detection accuracy on real source code. The recurrence notes indicate exploratory work across ten locations, but the saved table needs its missing source identifiers and evidence restored before the audit can be treated as reproducible. No additional vulnerability is claimed as confirmed.

The first milestone is therefore supported: scope is firmed up, target OSS is accessible, and an initial pipeline is running. The central next step is not to overstate the current prototype, but to strengthen its evidence: source-pinned recurrence records, real-source independent benchmark cases, stricter handling of each state invariant, and additional runtime validation where feasible.

Chapter A

Quick Explanation for the Progress Meeting

A 30-second explanation

We are studying whether known OpenSSL vulnerabilities can teach us reusable rules for detecting similar state and resource-lifecycle mistakes. We shortlisted eight CVEs using patch availability, root-cause clarity, typestate fit, recurrence potential, and time feasibility. We selected BIO-chain cleanup, PEM output-pointer cleanup, and TLS read-buffer lifetime cases. We wrote three targeted rules and tested them on vulnerable/fixed source revisions. The combined source-pattern regression passed for all three pairs; one case also has AddressSanitizer runtime evidence. We still need independent real-source evaluation and a reproducible recurrence audit before making claims about general accuracy or new vulnerabilities.

A 2-minute explanation

The idea is to turn a known bug and its fix into a statement about an invariant. In CVE-2023-0215, a BIO chain has to be restored before the filter is freed on an error path. In CVE-2022-4450, output pointers must not keep referring to buffers after those buffers are freed. In CVE-2024-4741, buffer release must be consistent with the record-layer packet pointer and length. We maintain vulnerable and fixed worktrees, implement a small Python check for each pattern, and run regression scripts that expect an alert on the vulnerable source and no alert on the fixed source. The result is a running initial pipeline, not a complete static analysis tool. Our separate 16-case benchmark is hand-written and therefore only a preliminary test of the simplified rules. The next work is to rebuild the recurrence evidence from exact source locations and evaluate on independent real-source examples.

Chapter B

Glossary

Term	Meaning in this project
API contract	Expected ownership, state, and return-value behaviour of a function.
ASan	AddressSanitizer, a runtime tool used here to detect memory safety issues during execution.
CVE	Public identifier for a disclosed vulnerability.
False positive (FP)	A non-vulnerable case reported as an alert.
False negative (FN)	A vulnerable case missed by the checker.
Invariant	A property that must hold for safe execution.
Regression pair	A vulnerable version and its corresponding fixed version, tested together.
Typestate	Model in which valid operations depend on the current state of an object.
UAF	Use-after-free: memory is accessed after its allocation has been released.
Vulnerable/fixed worktree	Separate checkout used to test source before and after a patch.

Chapter C

Official and Project References

[1] OpenSSL Security Advisory dated 7 February 2023, including CVE-2023-0215 and CVE-2022-4450: `https://www.openssl.org/news/secadv/20230207.txt`.

[2] OpenSSL vulnerabilities index: official index page.

[3] OpenSSL 3.0 vulnerability list, including CVE-2022-4450: official 3.0 series page.

[4] OpenSSL 3.3 vulnerability list, including CVE-2024-4741: official 3.3 series page.

[5] OpenSSL 4.0 vulnerability list, including CVE-2026-45447 and CVE-2026-35188: official 4.0 series page.

[6] OpenSSL fix commit for CVE-2023-0215: 8818064ce3c3c0f1b740a5aaba2a987e75bfbafd.

[7] OpenSSL fix commit recorded for CVE-2022-4450: fix commit 63bcf189....

[8] OpenSSL fix commit recorded for CVE-2024-4741: fix commit 2d059590....

[9] Project candidate register: `cve_candidates_openssl_v2.csv`.

[10] Project written rule models: `typestate_rules_spec_v2.md`.

[11] Project miniature-snippet benchmark and recorded output: `typestate_checker_eval_v2.py` and `eval_v2_stdout.log`.

[12] Project recurrence notes: `recurrence_investigation_v2.md`; source identifiers in the saved file require repair before the audit is fully reproducible.

Chapter D

Final Checklist Before Submission

- ☐ Replace the team and roll-number placeholders on the title page.
- ☐ Regenerate the source-pinned recurrence table and review all candidate classifications.
- ☐ Re-run the checker and tests from the exact working tree that will be shared.
- ☐ Include the final ‘git rev-parse HEAD‘ outputs and exact checkout instructions in the README.
- ☐ Keep synthetic snippet metrics separate from real-source/runtime metrics in the final paper and presentation.
- ☐ Do not claim a new vulnerability unless manual validation supports it.
- ☐ Confirm that the public repository contains no secrets, local paths that should be private, or unlicensed files.
- ☐ Prepare an Overleaf link, compiled PDF, public GitHub repository, and reproducibility instructions for the final deadline.